

HDOC DAY-39

Title: Variadic Functions & Command-Line Argument Processing

1. Problem Statements

1. Problem 1 (Variadic Function):

Implement a variadic function `double weightedAverage(int count, ...)` accepting count parameter followed by pairs of values and weights. Validate non-negative weights, prevent zero total weights, and compute the weighted average using `` ``.

2. Problem 2 (Command-Line Processing):

Accept arbitrary integers via command-line arguments. Validate and skip non-integer arguments, order valid numbers ascendingly, and compute minimum, maximum, average, median, and second-largest distinct value.

2. Theoretical Background & Logic

1. Variadic Functions (`stdarg.h`):

Variadic functions accept variable numbers of arguments. `va_list` holds argument data, `va_start` initializes traversal using fixed parameter count, `va_arg` retrieves each

parameter sequentially by type (double), and va_end cleans up argument pointer memory.

$$\text{Weighted Average} = \frac{\sum (v_i \cdot w_i)}{\sum w_i}$$

2. Command-Line Parsing & Numerical Analysis:

- argc and argv capture execution parameters.
- String validation ensures each character is a digit or leading signed symbol before conversion via atoi().
- Sorting simplifies minimum (arr[0]), maximum (arr[n-1]), and median calculation (middle value or midpoint average of middle pair).
- Second-largest distinct value search iterates backwards from n-2 until finding the first value strictly less than arr[n-1].

3. Test Case Tables

Problem 1 Test Cases

Test Case	Inputs	Expected Output	Status
1	weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)	84.00	Pass

Test Case	Inputs	Expected Output	Status
2	weightedAverage(2, 50.0, -0.5, 60.0, 0.5)	Error: Weight cannot be negative	Pass
3	weightedAverage(2, 50.0, 0.0, 60.0, 0.0)	Error: Total weight cannot be zero	Pass

Problem 2 Test Cases

Input Command	Expected Output Highlights	Status
./program 12 5 abc 8 12 20 3	Ignored: abc	

Ascending: 3 5 8 12 12 20

Min: 3, Max: 20, Avg: 10.00, Median: 10.00, 2nd Max: 12 |
Pass |

4. Source Code Listings

Problem 1 Source Code

```
#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...) {
    if (count <= 0) {
        printf("Error: Pair count must be greater than zero.\n");
        return -1.0;
    }

    va_list args;
    va_start(args, count);

    double total_weighted_sum = 0.0;
    double total_weight = 0.0;
    int is_valid = 1;

    for (int i = 0; i < count; i++) {
        double value = va_arg(args, double);
        double weight = va_arg(args, double);

        if (weight < 0) {
            printf("Error: Weight cannot be negative (Found weight = %.2f).\n",
weight);
            is_valid = 0;
            break;
        }

        total_weighted_sum += (value * weight);
        total_weight += weight;
    }

    va_end(args);
```

```

    if (!is_valid) return -1.0;

    if (total_weight == 0.0) {
        printf("Error: Total weight cannot be zero.\n");
        return -1.0;
    }

    return total_weighted_sum / total_weight;
}

int main(void) {
    printf("--- Test Case 1: Valid Input ---\n");
    double result1 = weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);
    if (result1 >= 0) {
        printf("Weighted Average = %.2f\n\n", result1);
    }

    printf("--- Test Case 2: Negative Weight Error ---\n");
    weightedAverage(2, 50.0, -0.5, 60.0, 0.5);
    printf("\n");

    printf("--- Test Case 3: Zero Total Weight Error ---\n");
    weightedAverage(2, 50.0, 0.0, 60.0, 0.0);

    return 0;
}

```

Problem 2 Source Code

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <stdbool.h>

bool is_valid_integer(const char *str) {
    if (str == NULL || *str == '\0') return false;
    int i = 0;
    if (str[0] == '-' || str[0] == '+') {
        if (str[1] == '\0') return false;
    }
}

```

```

    i = 1;
}
for (; str[i] != '\0'; i++) {
    if (!isdigit(str[i])) return false;
}
return true;
}

void bubble_sort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

double calculate_median(int arr[], int n) {
    if (n % 2 != 0) return (double)arr[n / 2];
    return (arr[(n / 2) - 1] + arr[n / 2]) / 2.0;
}

int get_second_largest_distinct(int arr[], int n, bool *found) {
    int largest = arr[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        if (arr[i] < largest) {
            *found = true;
            return arr[i];
        }
    }
    *found = false;
    return -1;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s ...\n", argv[0]);
    }
}

```

```

    return 1;
}

int *valid_numbers = (int *)malloc((argc - 1) * sizeof(int));
int valid_count = 0;

for (int i = 1; i < argc; i++) {
    if (is_valid_integer(argv[i])) {
        valid_numbers[valid_count++] = atoi(argv[i]);
    } else {
        printf("Invalid argument ignored: %s\n", argv[i]);
    }
}

if (valid_count == 0) {
    printf("Error: No valid integers provided.\n");
    free(valid_numbers);
    return 1;
}

bubble_sort(valid_numbers, valid_count);

printf("Valid integers in ascending order:\n");
for (int i = 0; i < valid_count; i++) {
    printf("%d ", valid_numbers[i]);
}
printf("\n");

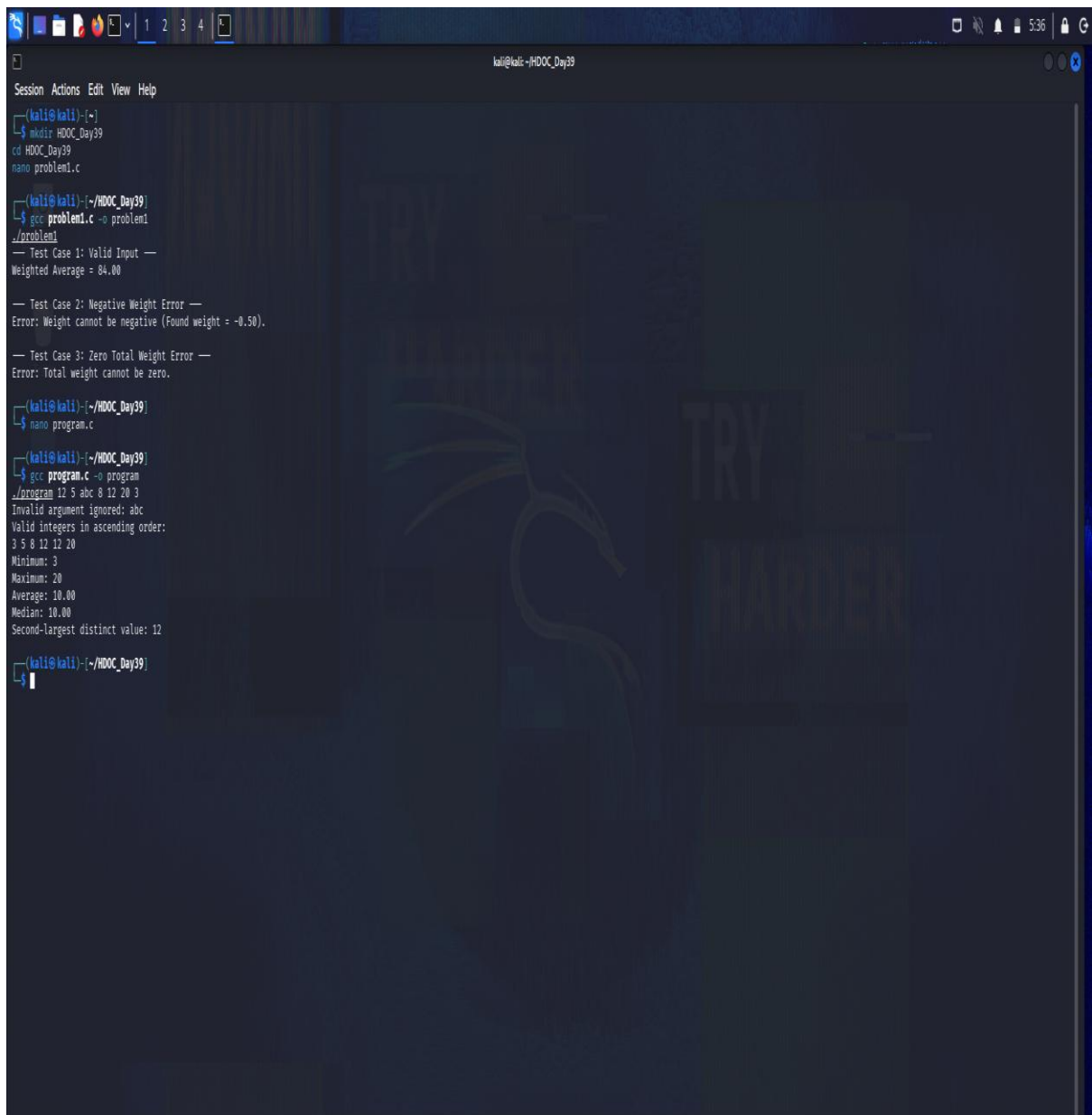
printf("Minimum: %d\n", valid_numbers[0]);
printf("Maximum: %d\n", valid_numbers[valid_count - 1]);
printf("Average: %.2f\n", (double)sum_array(valid_numbers,
valid_count) / valid_count);
printf("Median: %.2f\n", calculate_median(valid_numbers,
valid_count));

bool second_found = false;
int second_largest = get_second_largest_distinct(valid_numbers,
valid_count, &second_found);
if (second_found) {
    printf("Second-largest distinct value: %d\n", second_largest);
}

```

```
} else {  
    printf("Second-largest distinct value: None\n");  
}  
  
free(valid_numbers);  
return 0;  
}
```

5. Execution Screenshots



```
kali@kali:~/HDOOC_Day39  
$ mkdir HDOOC_Day39  
$ cd HDOOC_Day39  
$ nano problem1.c  
$ gcc problem1.c -o problem1  
$ ./problem1  
— Test Case 1: Valid Input —  
Weighted Average = 84.00  
  
— Test Case 2: Negative Weight Error —  
Error: Weight cannot be negative (Found weight = -0.50).  
  
— Test Case 3: Zero Total Weight Error —  
Error: Total weight cannot be zero.  
  
$ nano program.c  
$ gcc program.c -o program  
$ ./program 12 5 abc 8 12 20 3  
Invalid argument ignored: abc  
Valid integers in ascending order:  
3 5 8 12 12 20  
Minimum: 3  
Maximum: 20  
Average: 10.00  
Median: 10.00  
Second-largest distinct value: 12  
$
```